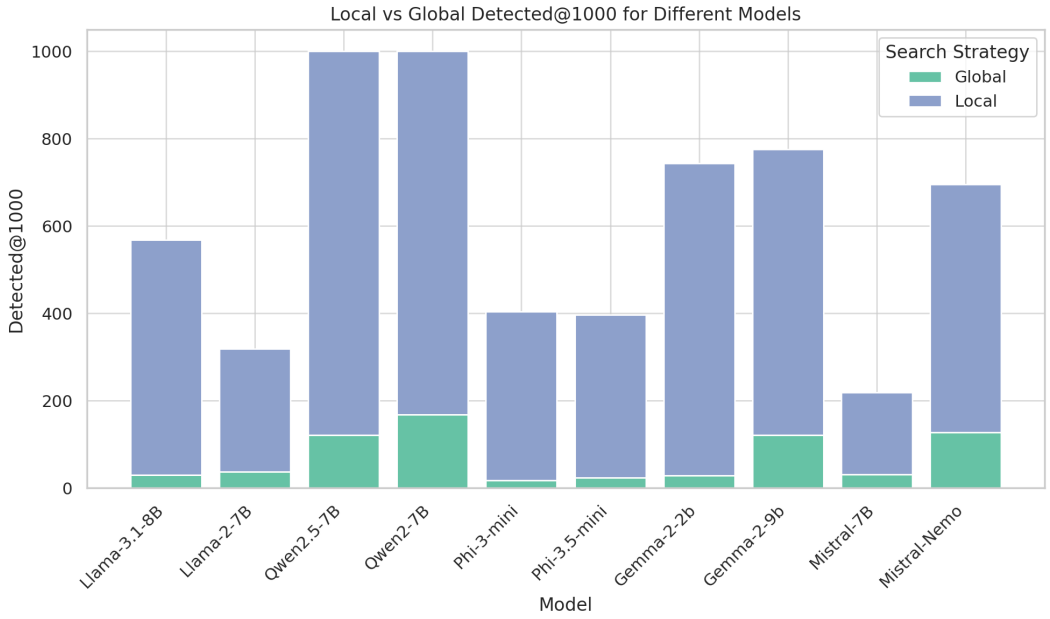


Figure 6: Comparison of GlitchMiner performance with and without local search strategy

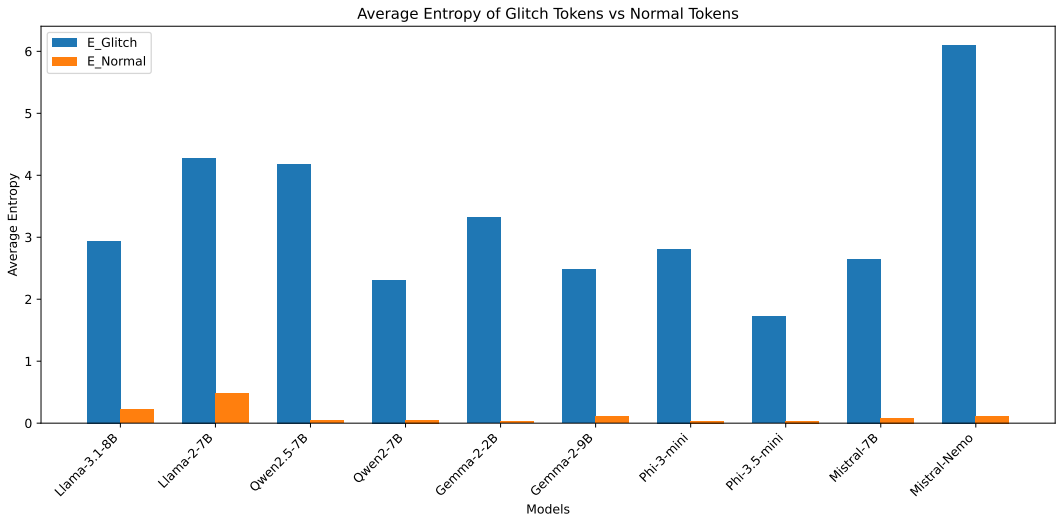


Figure 7: Average entropy comparison between glitch tokens and normal tokens across different models. Glitch tokens have higher entropy, indicating greater uncertainty in the model's predictions for these tokens.

within the model.

## 5 Conclusion

In this paper, we introduced GlitchMiner, a novel framework that detects glitch tokens in LLM by reframing the problem as a behavior-driven optimization task. Departing from prior work that relies on static patterns, our approach identifies anomalous tokens by searching for inputs that maximize the model's predictive uncertainty, measured by output entropy. We operationalize this principle with an efficient gradient-guided local search strategy that can accurately pinpoint high-entropy candidates in the vast token space. Extensive experiments on 10 diverse LLMs demonstrated that GlitchMiner significantly outperforms state-of-the-art baselines in both detection accuracy and efficiency. Our work provides a more robust and generalizable tool for auditing and enhancing the reliability of LLMs, with future work pointing towards mitigation strategies and adaptation for black-box models.

## Acknowledgments

We would like to express our sincere gratitude to Sander Land for his valuable insights and feedback on this work, particularly regarding the evaluation metrics. We wish to thank Guanlin Li for his valuable suggestions on our manuscript. This work was supported in part by the National Key R&D Program of China (2023YFB3107505), in part by Shaanxi Natural Science Funds for Distinguished Young Scholars(2023-JC-JQ-52), in part by the Natural Science Foundation of China (62302371), in part by the Postdoctoral Fellowship Program of CPSF (GZC20232035), and in part by the China Postdoctoral Science Foundation (2025M771552).

## References

Abdin, M.; Jacobs, S. A.; Awan, A. A.; Aneja, J.; Awadallah, A.; Awadalla, H.; Bach, N.; Bahree, A.; Bakhtiari, A.; Behl, H.; et al. 2024. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*.

AI, M. 2024a. Introducing Llama 3.1: Our most capable models to date. https://ai.meta.com/blog/meta-llama-3-1/. Accessed: 2024-10-18.

AI, M. 2024b. Mistral Nemo: Advancing the Capabilities of Large Language Models. https://mistral.ai/news/mistral-nemo/. Accessed: 2024-10-18.

Alibaba. 2024. Qwen 2.5: Advancing AI for Everyone. https://qwen2.org/qwen2-5/. Accessed: 2024-10-18.

Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H. P. D. O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Ebrahimi, J.; Rao, A.; Lowd, D.; and Dou, D. 2017. Hotflip: White-box adversarial examples for text classification. *arXiv preprint arXiv:1712.06751*.

Geiping, J.; Stein, A.; Shu, M.; Saifullah, K.; Wen, Y.; and Goldstein, T. 2024. Coercing LLMs to do and reveal (almost) anything. *arXiv preprint arXiv:2402.14020*.

Goel, A.; Gueta, A.; Gilon, O.; Liu, C.; Erell, S.; Nguyen, L. H.; Hao, X.; Jaber, B.; Reddy, S.; Kartha, R.; et al. 2023. Llms accelerate annotation for medical information extraction. In *Machine Learning for Health (ML4H)*, 82–100. PMLR.

Jiang, A. Q.; Sablayrolles, A.; Mensch, A.; Bamford, C.; Chaplot, D. S.; Casas, D. d. l.; Bressand, F.; Lengyel, G.; Lample, G.; Saulnier, L.; et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825*.

Jiang, J.; Wang, F.; Shen, J.; Kim, S.; and Kim, S. 2024. A Survey on Large Language Models for Code Generation. *arXiv preprint arXiv:2406.00515*.

Jury, B.; Lorusso, A.; Leinonen, J.; Denny, P.; and Luxton-Reilly, A. 2024. Evaluating llm-generated worked examples in an introductory programming course. In *Proceedings of the 26th Australasian Computing Education Conference*, 77–86.

Land, S.; and Bartolo, M. 2024a. Fishing for Magikarp: Automatically Detecting Under-trained Tokens in Large Language Models. *arXiv preprint arXiv:2405.05417*.

Land, S.; and Bartolo, M. 2024b. Fishing for Magikarp: Automatically Detecting Under-trained Tokens in Large Language Models. *arXiv preprint arXiv:2405.05417*.

LessWrong Community. 2023. SolidGoldMagikarp (plus, prompt generation). https://www.lesswrong.com/posts/aPeJE8bSo6rAFoLqg/so. Accessed: 2023-09-25.

Li, Y.; Liu, Y.; Deng, G.; Zhang, Y.; Song, W.; Shi, L.; Wang, K.; Li, Y.; Liu, Y.; and Wang, H. 2024. Glitch tokens in large language models: categorization taxonomy and effective detection. *Proceedings of the ACM on Software Engineering*, 1(FSE): 2075–2097.

Nijkamp, E.; Pang, B.; Hayashi, H.; Tu, L.; Wang, H.; Zhou, Y.; Savarese, S.; and Xiong, C. 2022. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474*.

Shen, X.; Chen, Z.; Backes, M.; Shen, Y.; and Zhang, Y. 2023. " do anything now": Characterizing and evaluating in-the-wild jailbreak prompts on large language models. *arXiv preprint arXiv:2308.03825*.

Shin, T.; Razeghi, Y.; Logan IV, R. L.; Wallace, E.; and Singh, S. 2020. Autoprompt: Eliciting knowledge from language models with automatically generated prompts. *arXiv preprint arXiv:2010.15980*.

Team, G.; Riviere, M.; Pathak, S.; Sessa, P. G.; Hardin, C.; Bhupatiraju, S.; Hussenot, L.; Mesnard, T.; Shahriari, B.; Ramé, A.; et al. 2024. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*.

Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

Traag, V. A.; Waltman, L.; and Van Eck, N. J. 2019. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific reports*, 9(1): 1–12.

Wang, S.; Xu, T.; Li, H.; Zhang, C.; Liang, J.; Tang, J.; Yu, P. S.; and Wen, Q. 2024. Large language models for education: A survey and outlook. *arXiv preprint arXiv:2403.18105*.

Wang, Y.; Ma, X.; and Chen, W. 2023. Augmenting black-box llms with medical textbooks for clinical question answering. *arXiv preprint arXiv:2309.02233*.

Wen, Y.; Jain, N.; Kirchenbauer, J.; Goldblum, M.; Geiping, J.; and Goldstein, T. 2024. Hard prompts made easy: Gradient-based discrete optimization for prompt tuning and discovery. *Advances in Neural Information Processing Systems*, 36.

Yang, A.; Yang, B.; Hui, B.; Zheng, B.; Yu, B.; Zhou, C.; Li, C.; Li, C.; Liu, D.; Huang, F.; et al. 2024. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*.

Zhang, Z.; Bai, W.; Li, Y.; Meng, M. H.; Wang, K.; Shi, L.; Li, L.; Wang, J.; and Wang, H. 2024. GlitchProber: Advancing Effective Detection and Mitigation of Glitch Tokens in Large Language Models. *arXiv preprint arXiv:2408.04905*.

Zou, A.; Wang, Z.; Carlini, N.; Nasr, M.; Kolter, J. Z.; and Fredrikson, M. 2023. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*.

# A Experimental Details

## A.1 Verification Templates

As stated in the main text, our robust verification process relies on three distinct templates for cross-validation to ensure reliability. A token is confirmed as a true glitch only if it fails the repetition task across all three. The first template is the one used during the optimization phase, while the other two are adapted from prior work to test for robustness under different phrasings. The structure of all three templates is detailed below for clarity and reproducibility.

**Template 1: Primary Optimization Template** **Description:** This is the primary template used to calculate output entropy during the gradient-guided search.

> **User Prompt:** `Please repeat the string: "{token}"`
> **Assistant Prefill:** `Sure, the string is: "‘`

**Template 2: GlitchHunter Adaptation (Direct Repetition)** **Description:** This template, adapted from GlitchHunter, directly asks the model to repeat the target token in a simple question-answer format.

> **User Prompt:** `Question: Please can you repeat the string '{token}'?`
> **Assistant Prefill:** `Of course! Here is the repeated string:‘`

**Template 3: Magikarp Adaptation (Few-shot Consistency)** **Description:** Adapted from Magikarp, this is a more complex few-shot template that first establishes a persona of a ”helpful assistant.” It tests if the model remains consistent when presented with the target token by first providing an example of how it handles a nonsensical string.

> **System Prompt:** `This helpful assistant explains what a phrase means, replying with identical responses to identical questions for reliability.`
>
> **Conversation History:**
> *User:* `What does 'edslfkj' mean?`
> *Assistant:* `'edslfkj' has no specific meaning I am aware of.`
>
> **Final User Prompt:** `What does '{token}' mean?`
> **Assistant Prefill:** ‘

## A.2 Analysis of the Verification Stage

The GlitchMiner framework operates as a two-stage pipeline:

1. A gradient-guided search proposes candidate tokens that maximize output entropy.
2. A robust verification stage confirms whether these candidates are true glitches.

The verification stage is intentionally stringent, requiring a candidate to fail the repetition task across **all three** prompt templates (including those adapted from GlitchHunter and Magikarp) to be confirmed. A token that fails on only one or two templates does not meet this strict criterion.

Table 4 summarizes the number of false-positive glitch tokens among the top 1000 candidates, which fail verification with template 1 but pass verification with either template 2 or template 3. This analysis demonstrates the necessity of employing cross-validation across multiple prompt templates.

## A.3 Rationale and Details of the Token Filter

The core function of our Token Filter is to determine a token's practical relevance and, consequently, its real-world risk profile. It answers a fundamental question: **can a given token in the vocabulary be triggered by a user through standard text input?** If a token cannot be produced from text, it is excluded from our analysis. This is not part of the glitch detection logic itself, but a crucial prerequisite that allows GlitchMiner to focus its computational budget efficiently on tokens that represent genuine security and reliability concerns.

Our filter removes three main categories of such tokens:

**SPECIAL Tokens** These are predefined functional symbols used for model structure or training (e.g., `[BOS]`, `</s>`). As they are not part of natural language inputs, they are straightforward to exclude.

**Unreachable Tokens** These are tokens that exist in the vocabulary but can never be generated by the model's tokenizer from any text input, often existing as artifacts of the tokenizer's construction. We provide a concrete example from the `Llama-2-7B-chat-hf` model to illustrate this:

- The token ID `71` corresponds to the bytecode `\x44`, which the tokenizer decodes into the string “D”.
- However, when we re-encode either the string “D” or the bytecode `\x44`, the tokenizer consistently produces the token ID `360`, never 71.

This demonstrates that no text input a user can provide will ever result in token 71. Since it cannot be triggered in practice, testing it for ”glitchiness” is unnecessary.

**Undecodeable Tokens** This category includes tokens corresponding to byte sequences that cannot be decoded into a valid string, often due to violating encoding standards like UTF-8. For example, in the same Llama-2 tokenizer:

- The token ID `255` corresponds to the byte `\xFC`.
- This byte is an invalid start for a UTF-8 character sequence. Therefore, it cannot be decoded into coherent text and often appears as a generic replacement symbol.

Such tokens are excluded as they do not map to any meaningful user-generated text.

# B Examples of Discovered Glitch Tokens

Table 3 showcases several examples of glitch tokens discovered by GlitchMiner across different models, illustrating the variety of anomalous behaviors that can be triggered. The ”Response” column displays each model's full output when given the primary verification prompt, `Please repeat the string: {token}`.